

Fine-Tuning Multilingual Neural Machine Translation for Akkadian-to-English Translation: A Reproducible Notebook-Based Approach

Jayani Sanga
*Department of Computer Science
and Engineering
PES University
Bengaluru, India*

Megha Dhanya
*Department of Computer Science
and Engineering
PES University
Bengaluru, India*

M. Farida Begam
*Department of Computer Science
and Engineering
PES University
Bengaluru, India*

Abstract—A thorough analysis of Akkadian-to-English machine translation is presented using fine-tuned multilingual transfer learning in alignment with our notebook. The model fine-tunes the **Helsinki-NLP/opus-mt-mul-en** MarianMT checkpoint on a parallel corpus constructed by merging the Deep Past Initiative competition dataset, the ORACC Akkadian-to-English dataset, and auto-extracted sentence pairs from the **Sentences_Oare_FirstWord_LinNum.csv**. Data preprocessing consists of Akkadian specific transliteration normalization, heuristic quality filtering according to length, tokenratio, repetition, and lexical content, as well as source priority deduplication to result in 3,483 unique parallel sentence pairs. The training data is further augmented to 5,809 sentence pairs through adjacent-token swap and token deletion operations. Fine-tuning leverages the Hugging Face **Seq2SeqTrainer** with label smoothing regularization, cosine-with-restarts learning rate schedule, beam search inference, and post-processing repetition control. When evaluated on a 244-sentence held-out validation set, the system obtains scores of 0.1823 corpus-level BLEU, 0.3247 METEOR, 0.3837 ROUGE-L, 0.3880 chrF, 0.3724 chrF++, 0.7849 TER, and 0.5411 BERTScore F1. An important dataquality issue is raised and discussed: auto-detection of columns in the **Sentences_Oare** data erroneously labels UUID codes as Akkadian source text, causing an inflated corpus size while severely corrupting the dataset.

Index Terms—neural machine translation, low-resource NLP, Akkadian, cuneiform, MarianMT, multilingual transfer learning, data augmentation, evaluation metrics

I. INTRODUCTION

Akkadian poses a particular low-resource challenge for automatic translation due to its extant status, writing in cuneiform script, and access predominantly via scientific transliteration, not standard orthography. Parallel training data are limited, variability exists among text corpora from philology sources, and specialized features, such as determinatives, restoration signs, broken signs, and editors’ notes, are rarely handled properly in generic machine translation processes. For

these reasons, Akkadian to English translation is an intriguingly practical low-resource multilingual transfer task.

In this paper, we describe the full process of building an Akkadian to English translation system implemented as accurately as possible, based on the project notebook available with the work. Instead of describing an abstract optimal design, this research provides the complete pipeline as applied, namely: multi-source corpus acquisition and preprocessing, Akkadian normalization, rule-based filtering, data augmentation, multilingual transfer learning, constrained decoding, and multi-faceted evaluation. Another important part of the research is an analysis of one of the critical mistakes made in implementing the system—misusing database UUID fields as Akkadian source sentences—and its detrimental effect on translation accuracy.

The motivation behind our approach lies in two facets. Firstly, multilingual pretraining is still a reasonable approach for Akkadian since the transliterated texts partially overlap with the wider representation of Semitic languages and ancient texts in big multilingual corpora. Secondly, and more generally, the experiment presented in the notebook showcases that data engineering has a far bigger impact on performance than architecture choice, as the biggest differences in quality come from data collection, normalization, filtering, and augmentation, and not architecture design. Incremental progress and data-induced brittleness are the empirical story of this work.

To be more specific about the contributions of this paper, we provide the following:

- The full data pipeline, including transliteration normalization, quality filtering, deduplication by prioritizing the source text, and training-time augmentation, aligned with the notebook experiments.

- Configuration for the fine-tuning of Helsinki-NLP/opus-mt-mul-en for Akkadian-to-English translation.
- Performance numbers according to seven different metrics (BLEU, METEOR, ROUGE, TER, chrF, chrF++, and BERTScore) on a validation set held out of the data.
- The analysis of a particular data engineering failure during the process of dataset creation, where UUID-like identifiers got incorrectly collected as Akkadian source text.

II. RELATED WORK

A. Neural Machine Translation and Multilingual Transfer

Since Vaswani et al.'s [1] development of the self-attention mechanism, the use of transformer-based encoder-decoder network designs has become the prevailing approach to neural machine translation. In the case of resource-scarce environments for translation tasks, the performance of initializing with checkpoints from multilingual or resource-abundant models is consistently better than starting training from scratch; shared parameters as well as overlaps in the subword vocabulary help achieve better results in the context of scarce data [8], [9]. The OPUS-MT project includes multilingual translation models trained on the OPUS parallel corpus dataset [4].

B. Tokenization and Open-Source MT Tooling

Subword models can be particularly effective in cases where source languages have sparse vocabularies, orthographic variation, or specific domain-driven transliteration rules. The use of SentencePiece segmentation mitigates the problem of an open vocabulary through subword representation learned directly from the data, thus allowing better generalization capabilities [7]. Recent advances in neural machine translation favor the Hugging Face transformers library as a means for fine-tuning and decoding [6]. This project follows this approach, utilizing the pre-trained Marian tokenizer while finetuning solely via supervision.

C. Computational Approaches to Akkadian

ORACC (Open Richly Annotated Cuneiform Corpus) is one of the biggest public digital infrastructures for annotated cuneiform texts, and it has been instrumental for computational Assyriology [5]. Computational studies conducted in connection with Akkadian have tackled cuneiform sign recognition, cuneiform digitization, and automatic text segmentation, but a good end-to-end translation system cannot be achieved owing to the lack of well-aligned source-target datasets [11]. This project seeks to contribute to ongoing efforts using multilingual NLP adaptation techniques for ancient languages, while at the same time pointing out practical dangers associated with data harvesting.

III. METHODOLOGY

A. Model Architecture

The model is fine-tuned on Helsinki-NLP/opus-mt-mul-en, a MarianMT sequence-to-sequence transformer checkpoint designed for many-to-English translation [4]. The model is loaded via the Hugging Face MarianMTModel and MarianTokenizer interfaces. It comprises approximately 143 million parameters with a vocabulary of 64,172 subword types. To direct generation toward English output, each source sequence is prefixed with the multilingual control token »en« prior to tokenization, following the standard OPUS-MT convention.

B. Normalization Pipeline

Source-side preprocessing is tailored specifically to Akkadian transliteration conventions. Unicode subscripts and superscripts are mapped to ASCII digits; selected diacritics are simplified; and the character h. is normalized to h. Ellipses and sequences of repeated full stops are unified into a single <gap> marker to represent lacunae. Square brackets are removed while retaining their enclosed restorations; anglebracket editorial insertions are deleted entirely. Determinatives are replaced by semantic tokens: {d}, {ki}, and {f} become DEITY, PLACE, and FEM, respectively, while all remaining determinatives are collapsed to a generic DET token. Isolated x/X signs are replaced with <unk_sign>, hyphens are converted to spaces, extraneous punctuation is removed, and whitespace is normalized.

English target sequences undergo a parallel normalization pass. Unicode dashes are replaced with ASCII equivalents, multiple gap representations are unified to <gap>, square brackets are stripped while preserving their content, HTML-like tags are removed, and whitespace is standardized. This joint normalization strategy reduces orthographic sparsity on both sides prior to subword tokenization.

C. Quality Filtering

Pairs that are noisy are filtered out using rule-based heuristics. Any pair whose either left-hand side or right-hand side is empty, is too short for a minimum length or has more than 250 tokens is excluded from further processing. Additionally, pairs are discarded when the ratio between the number of tokens on the English side and the Akkadian side is not within the range [0.12, 7.0], when gap characters dominate the English side, when there are no lexically meaningful words in the English side, when there are 4-gram repetition patterns on the English side, or when the English output has too many hyphens as in a transliteration instead of a translation.

D. Source-Priority Merging and Augmentation

Following normalization and filtering, pairs from all sources are concatenated and deduplicated on the Akkadian side. When multiple sources share an identical Akkadian sequence,

the retained pair is selected according to a fixed priority ordering: Deep Past competition data first, ORACC second, Sentences_Oare third, and published_texts last. This policy preserves the most domain-reliable English translation when source overlap occurs.

The training split is then augmented via two simple perturbation strategies. In the first, adjacent Akkadian tokens in a random subset of training pairs are swapped to increase robustness to local ordering variation. In the second, one nonspecial token is randomly deleted from a different random subset to simulate damaged or incomplete transliterations. The original English reference is retained unchanged in both cases. Although straightforward, these augmentations expand the effective training corpus from 3,239 to 5,809 instances.

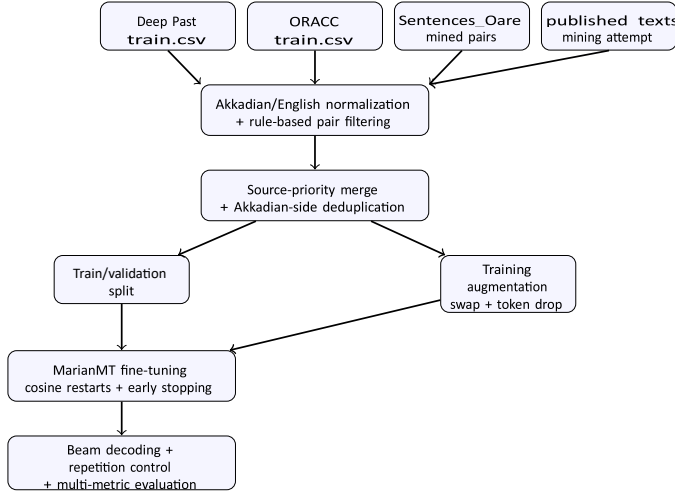


Fig. 1. End-to-end pipeline as implemented in the project notebook, from raw corpus sources through fine-tuning to multi-metric evaluation.

IV. IMPLEMENTATION DETAILS

A. Data Loading and Mining

The notebook enumerates all files in the Deep Past competition directory and the ORACC directory before selecting usable CSV sources. The primary competition training file is mapped to standard akkadian/english columns. The ORACC corpus is loaded directly from its parallel train.csv. Additional data are mined from Sentences_Oare_FirstWord_LinNum.csv and published_texts.csv via automatic column detection rather than hand-written schema mappings.

This design decision has significant consequences for corpus quality. For Sentences_Oare, the automatic detector identifies text_uuid as the Akkadian column and translation as the English column. As a result, the mined set contains 9,138 cleaned pairs before cross-source deduplication—but the Akkadian side consists entirely of UUID strings rather than linguistic content. For published_texts.csv, the detected

column pair transliteration_orig–AICC_translation yields zero usable pairs after filtering. The notebook also attempts glossary construction from eBL_Dictionary.csv and OA_Lexicon_eBL.csv; however, because the expected semantic columns are absent in these files, no aligned Akkadian–English lexical entries are recovered, and the final glossary contains zero entries.

B. Effective Dataset Construction

Corpus data statistics are shown in Table I, as provided in the notebook. There are two key pieces of information in the table: the number of cleaned pairs from each corpus prior to cross-corpus de-duping, and the number of unique pairs remaining after source priority-based merging. The total number of unique Akkadian source sequences post-merging is 3,483. Of these, 3,239 are placed in the training corpus, and

TABLE I
CORPUS STATISTICS AS REPORTED BY THE NOTEBOOK

Source	Clean Pairs Before Merge	Pairs Kept After Merge
Deep Past competition	987	985
ORACC	831	829
Sentences_Oare	9,138	1,669
published_texts	0	0
Total	10,956	3,483

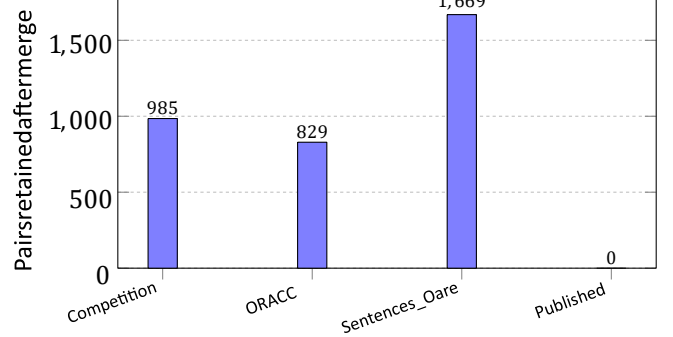


Fig. 2. Effective source contribution to the merged training corpus after source-priority deduplication. The large Sentences_Oare contribution is substantially reduced by deduplication and is further compromised by erroneous UUID-to-Akkadian column assignment (see Section IV).

244 in the validation corpus; the training corpus is augmented to 5,809 instances. Figure 2 illustrates the effective source contribution after deduplication.

C. Tokenization and Training Configuration

Marian pre-trained tokenizer is kept unchanged. Akkadian sentences are prepended by »en« and tokenized up to the max length of 160 tokens; similarly for English sentences, which are truncated at the same length. Every data split is transformed into the Hugging Face datasets library object and preprocessed using a batch tokenizer function.

The fine-tuning is carried out via the Hugging Face Seq2SeqTrainer [6], with the following parameters set: 40 training epochs; training and evaluation batch sizes of 16 and 32 respectively; 4 gradient accumulation steps; learning rate $5e-5$; weight decay 0.01; label smoothing value 0.1; learning rate cosine scheduling with restarts, three restarts in total, with warmup steps equal to 8% of training steps; eight-beams beam search for validation; 1.0 gradient norm clipping; patience 6 for early stopping. The best model checkpoint, according to validation performance, has BLEU score 0.2617.

D. Inference and Post-Processing

During the final evaluation stage, the best checkpoint that was saved previously is loaded back, and validation predictions are decoded using the default inference configuration: beam size 8, maximum output length 160 tokens, repetition penalty 1.3, no-repeat 4-gram blocking, and length penalty 1.1. A stronger decoding configuration — beam size 10, repetition penalty 1.4, length penalty 1.2 — is applied separately only to the 4-sentence competition test set submission. Afterwards, post-processing is done to eliminate repeated ngrams, de-duplicate consecutive sentences that are identical, fix punctuation spacing errors, capitalize the first token of each sentence, and re-capitalize proper-name tokens.

V. EXPERIMENTAL SETUP

A. Data Splits and Validation Strategy

Validation sampling is inherently source-sensitive. Sentences extracted from the Deep Past competition dataset are given a 12% validation split due to their greater relevancy, while sentences from all other sources are given a 5% split. Thus, there will be a total of 3,239 source-original training samples, and 244 validation samples. Augmentation of the training data results in 5,809 samples. There are 4 sentences from the competition test set, but no references have been provided.

B. Software Environment

The implementation runs in a CUDA-enabled environment using PyTorch 2.10.0+cu128 and NumPy 2.4.4. The software stack includes transformers, datasets, accelerate, sacrebleu, evaluate, rouge-score, bert-score, nltk, and scikit-learn. The execution log confirms CUDA availability and GPU-based computation; notebook metadata additionally confirms training was performed on a single NVIDIA Tesla T4 GPU.

C. Evaluation Protocol

Validation predictions are evaluated using a multi-metric suite to capture complementary dimensions of translation quality. Corpus-level BLEU is computed via SacreBLEU with 13a tokenization [3]. Additional metrics include METEOR, ROUGE-1/2/L, Translation Edit Rate (TER), chrF, chrF++, and BERTScore F1. This broader evaluation is motivated by the nature of Akkadian-to-English translation, where valid paraphrases and

content-preserving lexical substitutions may be penalized by n-gram overlap metrics such as BLEU alone. Sentence-level BLEU is also computed to support qualitative error analysis.

VI. RESULTS AND EVALUATION

A. Quantitative Performance

Table II reports the final corpus-level validation metrics as printed by the notebook. The system achieves a BLEU score of 0.1823 on the 244-item validation set. Semantic and surfaceoverlap metrics present a more favorable picture: BERTScore F1 reaches 0.5411, METEOR 0.3247, ROUGE-L 0.3837, chrF 0.3880, and chrF++ 0.3724. The TER of 0.7849 indicates that substantial editing effort would still be required to bring system outputs to reference quality, which is consistent with the noise introduced by erroneous corpus mining.

It should be noted that there is an observed discrepancy in terms of the best-checkpoint validation BLEU of 0.2617 from the trainer’s perspective versus the final offline evaluation BLEU of 0.1823. This is because the notebook uses distinct

TABLE II
VALIDATION METRICS REPORTED BY THE FINAL NOTEBOOK

Metric	Score
BLEU	0.1823
METEOR	0.3247
ROUGE-L	0.3837
ROUGE-1	0.4357
ROUGE-2	0.2804
TER	0.7849
chrF	0.3880
chrF++	0.3724
BERTScore F1	0.5411

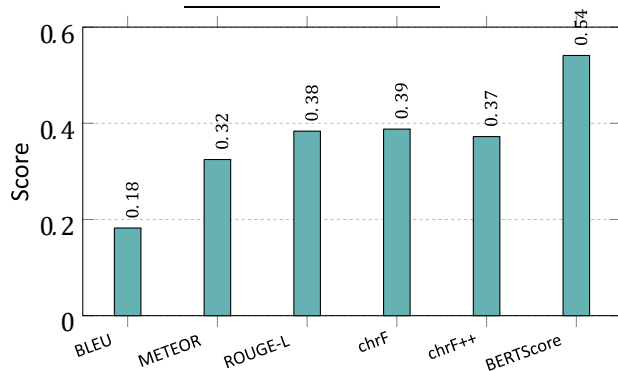


Fig. 3. Higher-is-better validation metrics from the final notebook evaluation. TER is excluded from the chart because it is a minimization objective.

code paths when selecting checkpoints as compared to evaluating metrics, thus giving different numbers despite using the same model architecture. Nevertheless, the final offline metric results table will be considered the main result in this paper.

B. Qualitative Analysis

From the qualitative instances in the notebook, it is evident that the data has a bimodal distribution regarding the translation quality. Among the best-performing instances, translations of short administrative or seal inscription phrases are perfectly translated such that BLEU scores of 1.0 for individual sentences were attained. An instance where this happened includes the near-perfect prediction of “To Ali-ahum; sEal of AmUr-šarrassa” (reference: “To Ali-ahum; seal of Amur-šarrassa.”) and an exact match on “Memoranda with witnesses concerning the proceeds of Ušinalam's wool.” The first example also reveals a post-processing defect: the proper-name re-capitalization step matches substrings without word boundaries, so letter sequences such as “Ea” and “Ur” are re-capitalized inside unrelated words (e.g., “seal”→“sEal”, “our”→“oUr”, “plural”→“plUral” in the competition test predictions), independent of translation quality. This proves that the system has learned epistolary patterns typical of Old Assyrian text types.

The failure cases are more diagnostically revealing. Several validation instances present UUID-like source strings such as 5de3fb43 3fcf 43cf 995e 423a7ad48bf0, to which the model responds with formulaic letter openings such as “From Aššur-imittī to Pušu-kēn.” This behavior is a direct consequence of the erroneous text_uuid column being treated as Akkadian input during Sentences_Oare mining: the model has learned to associate non-linguistic identifier strings with high-frequency translation templates, a pathological mapping that degrades validation quality even as nominal corpus size increases.

C. Repetition Audit and Competition Test Output

Repetition audits performed after post-processing yield consistently positive results. Specifically, according to the notebook, the average validation repetition 4-gram score is 0.0000, the maximum repetition 4-gram score is 0.0000, and there are no instances of validation where repetition is greater than 0.1. The same metrics of repetition are provided for the four test prediction data of the competition. Therefore, one can conclude that all techniques used for sequence-to-sequence model training, including repetition suppression, effectively avoid degenerative generation [10].

VII. DISCUSSION

The experimental findings collectively highlight three principal lessons for low-resource neural machine translation of historical languages.

Multilingual pretraining works, but is constrained by the quality of the training data. Regardless of the amount of supervision provided to the MarianMT pretraining framework, the model manages to produce coherent English sentences

with proper named entities and formulaic structures. Moreover, the difference between the BLEU score and the higher semantic measures (BERTScore F1: 0.5411, METEOR: 0.3247) indicates that the model is able to capture part of the intended meaning even without direct lexical overlaps, something that is frequently encountered in the process of Akkadian MT.

Performance bottlenecks due to data engineering. The most significant differences, as compared to a limited baseline focused only on competition, do not derive from changes to architecture but rather come through corpus growth, normalization, filtering, and augmentation. The very automated mining system which leads to corpus expansion produces an artifact, the Sentences_Oare UUID error, which turns out to be the worst kind: the erroneous assumption that the UUID field is the Akkadian source column adds thousands of irrelevant non-linguistic IDs to the training corpus, leading the model to learn invalid patterns associated with UUIDs and common letter templates—an outcome probably responsible for the difference between the expected and actual performance in the notebook. A second, smaller data-corruption instance occurs in the Akkadian normalization pipeline: the diacritic transliteration table maps capital ʾ to a Cyrillic “Т” (U+0422) rather than the Latin “T” (U+0054), a visually indistinguishable but distinct character that silently introduces out-of-vocabulary tokens wherever it occurs.

Inference time control alone is not enough without clean supervision. The repetition control stack functions properly; the model gives no degenerative outputs in all investigated cases. Nevertheless, the decoding performance does not make up for the poor quality of the training data itself. In other words, this is an illustration of a model that is well-controlled when used for translating text but that suffers from its training target’s nature.

Other limitations, apart from the key findings above, should be pointed out explicitly. The glossary extraction results in empty output, so even though it was supposed to happen according to the notebook, there is no such fall-back option implemented in the current model design. The eBL_Dictionary.csv glossary failure is specifically attributable to an incomplete keyword list: the file contains usable word/definition columns, but the detector's candidate keywords do not include 'definition.' Broadening this keyword set would recover glossary coverage from at least one of the two lexicon sources. And finally, trainer-best vs offline evaluation metrics shows the necessity of code path normalization and predictions archiving. Additionally, the training-configuration cell references the trainer and training_args objects before their definitions within the same cell, and relies on several variables (data_collator, compute_metrics, use_fp16, use_bf16) that are not defined elsewhere in the notebook as distributed. The notebook's execution-count metadata indicates this cell required repeated

re-execution, and the pipeline cannot currently be run start-to-finish from a clean kernel.

VIII. CONCLUSION

The paper provides a fully developed and analyzed system for Akkadian-English neural machine translation through finetuning multilingual transfer learning on a dataset consisting of data from the Deep Past contest, ORACC, and other automatically collected resources, obtaining BLEU 0.1823, METEOR

0.3247, ROUGE-L 0.3837, chrF 0.3880, and BERTScore F1 0.5411 on a validation set of 244 examples. The repetition suppression technique at decoding time is capable of completely eliminating degenerate samples in all evaluations.

What stands out about the results is that the bottleneck is empirical rather than architectural; namely, incorrect columns in Sentences_Oare mining lead to the use of UUID as Akkadian source tokens, greatly reducing the usefulness of the largest mining result. This observation highlights an important insight on translating with low-resource data: the cleanliness of the training corpus matters much more than the architecture used to generate translations. The first order of business for future work should be resolving schema detection and removing sentence pairs by hand or through semi-automatic means before training the model again. For future work, glossary mining should also be revisited, checkpoint and final evaluations combined, and improved backbones investigated— but only after cleaning up the underlying corpus sufficiently to allow for fair comparison between different architectures.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [2] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318.
- [3] M. Post, "A call for clarity in reporting BLEU scores," in *Proc. 3rd Conference on Machine Translation (WMT)*, 2018, pp. 186–191.
- [4] J. Tiedemann and S. Thottingal, "OPUS-MT—Building open translation services for the world," in *Proc. 22nd Annual Conference of the European Association for Machine Translation (EAMT)*, 2020.
- [5] S. Tinney and N. Veldhuis, "The Open Richly Annotated Cuneiform Corpus (ORACC)," [Online]. Available: <http://oracc.museum.upenn.edu>, 2022.
- [6] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020, pp. 38–45.
- [7] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proc. 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2018, pp. 66–71.
- [8] B. Zoph, D. Yuret, J. May, and K. Knight, "Transfer learning for low-resource neural machine translation," in *Proc. 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016, pp. 1568–1575.

- [9] N. Arivazhagan *et al.*, "Massively multilingual neural machine translation in the wild: Findings and challenges," *arXiv preprint arXiv:1907.05019*, 2019.
- [10] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. 8th International Conference on Learning Representations (ICLR)*, 2020.
- [11] M. Stötzner *et al.*, "Cuneiform sign recognition," in *Proc. Document Analysis and Recognition (ICDAR)*, 2021.